



***End-to-End Real-Time IoT Data Pipeline
Using Kafka, Spark, InfluxDB, and Grafana***

Institution: KFUPM

Instructor: Waleed AlGobi

By: Rahaf Abdullah Bajabaa

Student ID: g202512330

Abstract

This project presents a complete end-to-end real-time data processing pipeline designed for Internet of Things (IoT) applications. The system integrates Kafka for high-throughput message buffering, Spark Structured Streaming for real-time processing, InfluxDB for efficient time-series storage, and Grafana for dynamic visualization and alerting. The pipeline ingests continuous multi-metric sensor data, transforms it with near-zero latency, and visualizes it through real-time dashboards. The architecture was deployed using containerized services and demonstrates high scalability, resilience, and consistent performance. Experimental results confirm the successful ingestion, transformation, storage, and visualization of four key electrical parameters. Additional features such as multi-panel Grafana dashboards and threshold-based alerting further enhance system monitoring and analytical capabilities.

1. Introduction

The increasing adoption of IoT technologies in industrial automation, energy monitoring, and smart infrastructure demands robust real-time data processing pipelines. Traditional batch-based systems are inadequate for scenarios requiring immediate feedback, anomaly detection, and rapid decision-making.

This project implements a modern streaming architecture aligned with the ICS 574 Big Data Analytics course requirements. The pipeline collects voltage, current, active power, and apparent power readings from a sensor simulator, transmits them through Kafka, processes them using Spark Structured Streaming, stores them in InfluxDB, and visualizes the results through Grafana dashboards. The system mimics real-world industrial

IoT deployments and highlights the core components of scalable and fault-tolerant streaming systems.

2. System Architecture Overview

The system is composed of five major components that work together to provide continuous data flow from ingestion to visualization.

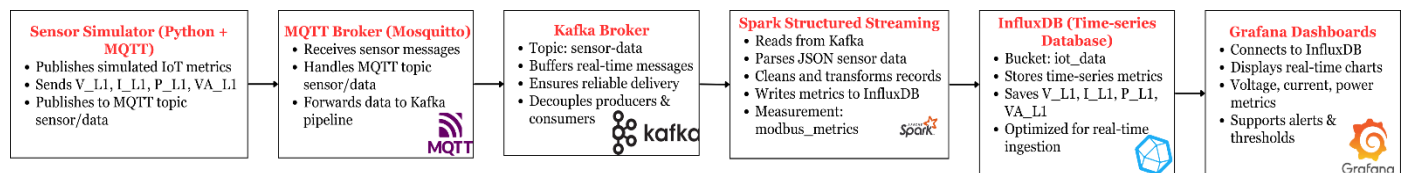


Figure 1 – Overall System Architecture Diagram

This figure illustrates the complete end-to-end IoT data pipeline, showing how sensor readings flow through MQTT, Kafka, Spark Structured Streaming, InfluxDB, and Grafana for real-time processing and visualization.

2.1 MQTT Sensor Simulator (Mosquitto)

A Python-based simulator generates periodic readings for the following metrics:

- **V_L1** – Line voltage
- **I_L1** – Line current
- **P_L1** – Active power
- **VA_L1** – Apparent power

These values are published every few seconds to the MQTT topic: sensor/data

```

MINGW64/c/bigdata-project/Real_TimeIoT_Monitoring_with_Kafka_Spark_InfluxDB_Grafana/modbus-simulator
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051741}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051743}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051745}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051747}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051749}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051751}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051753}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051755}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051757}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051759}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051761}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051763}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051765}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051767}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051769}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051771}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051773}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051775}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051777}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051779}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051781}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051783}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051785}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051787}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051789}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051791}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051793}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051795}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051797}
Published: {'device_id': 'sensor1', 'V_L1': 220, 'I_L1': 5, 'VA_L1': 1100, 'P_L1': 990.0, 'timestamp': 176
5051799}
Published: {'device_id': 'sensor1', 'V_L1': 230, 'I_L1': 6, 'VA_L1': 1380, 'P_L1': 1242.0, 'timestamp': 17
65051801}
Published: {'device_id': 'sensor1', 'V_L1': 240, 'I_L1': 7, 'VA_L1': 1680, 'P_L1': 1512.0, 'timestamp': 17
65051815}

```

Figure 2 – MQTT Publisher Output

This figure shows the real-time sensor readings generated by the MQTT-based simulator, including voltage, current, active power, apparent power, and timestamps published continuously to the topic sensor/data.

2.4 InfluxDB Time-Series Storage

A bucket named **iot_data** was created to store the incoming metrics. Each record is written under the measurement:

modbus_metrics

with tags and fields including:

- device_id
- V_L1
- I_L1
- P_L1
- VA_L1

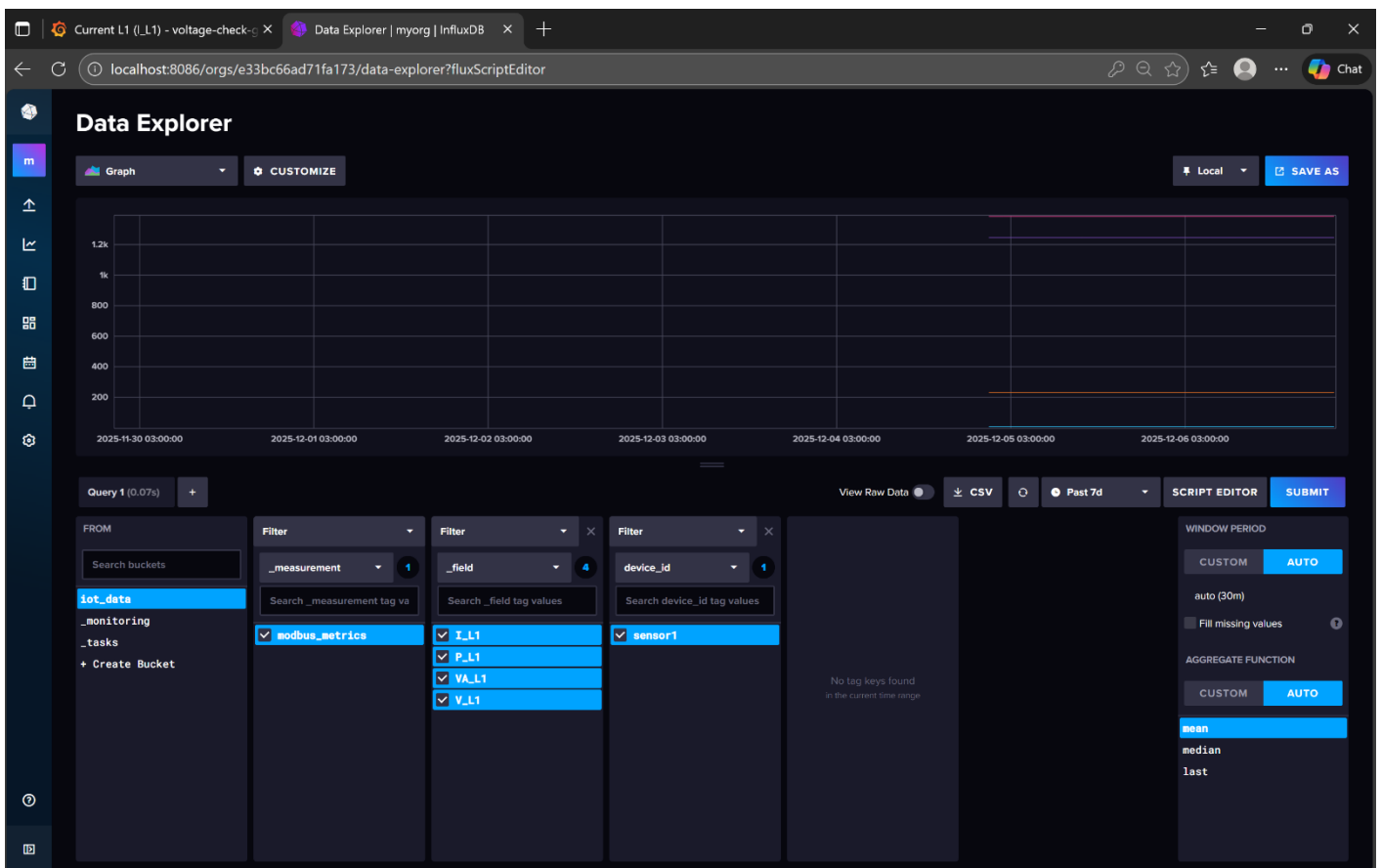


Figure 5 – InfluxDB Data Explorer Showing Live Metrics

This figure presents the real-time sensor metrics stored in the **iot_data** bucket and visualized through the InfluxDB Data Explorer, confirming correct ingestion of **V_L1**, **I_L1**, **P_L1**, and **VA_L1**.

2.5 Grafana Dashboard Visualization

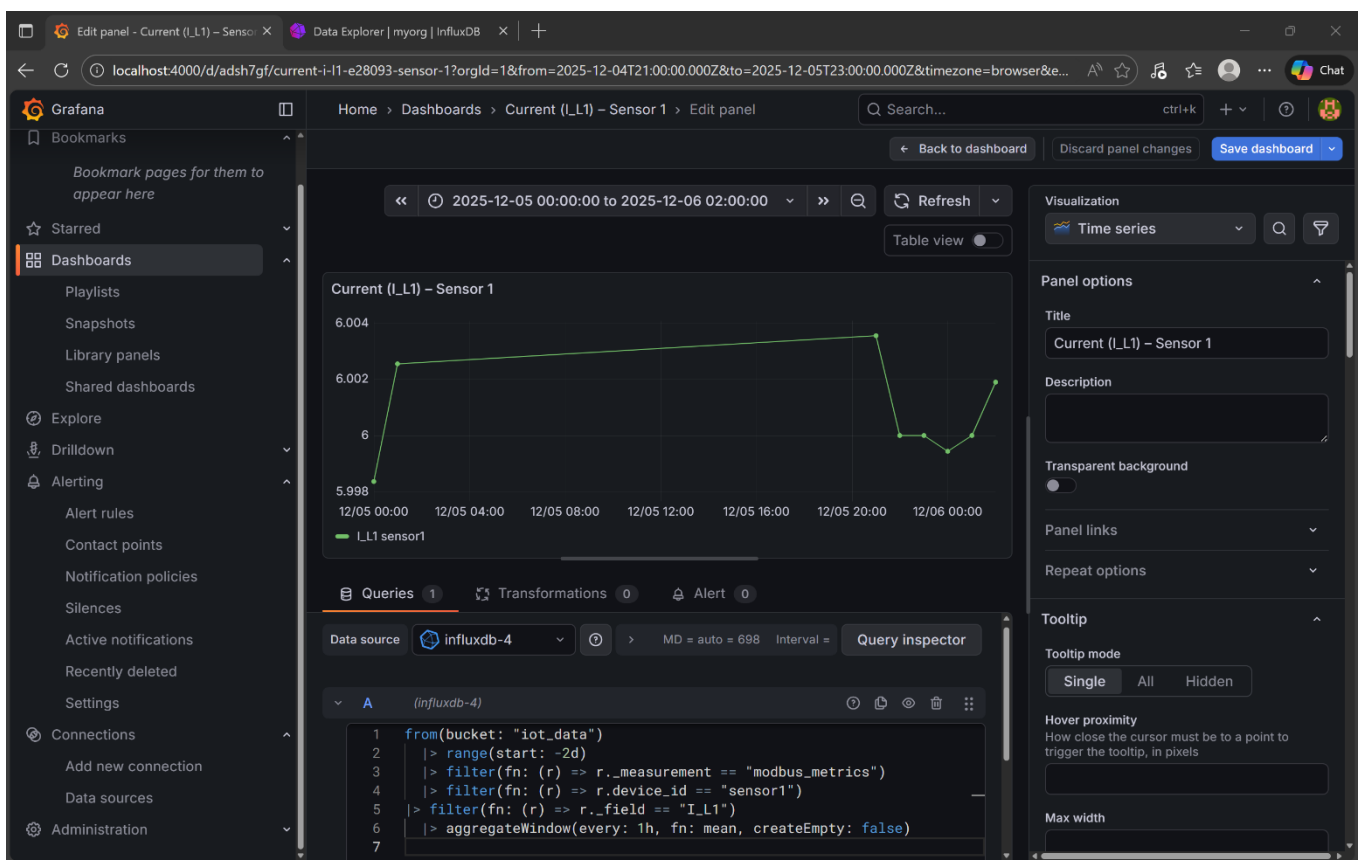
Grafana was configured with InfluxDB (Flux query mode) as a data source.

Real-time dashboards were built to display:

1. **Voltage (V_L1)**
2. **Current (I_L1)**
3. **Active Power (P_L1)**
4. **Apparent Power (VA_L1)**

Additional features implemented:

- ✓ Mean aggregation over continuous windows
- ✓ Trend markers
- ✓ Multi-panel layout
- ✓ Threshold-based alerting



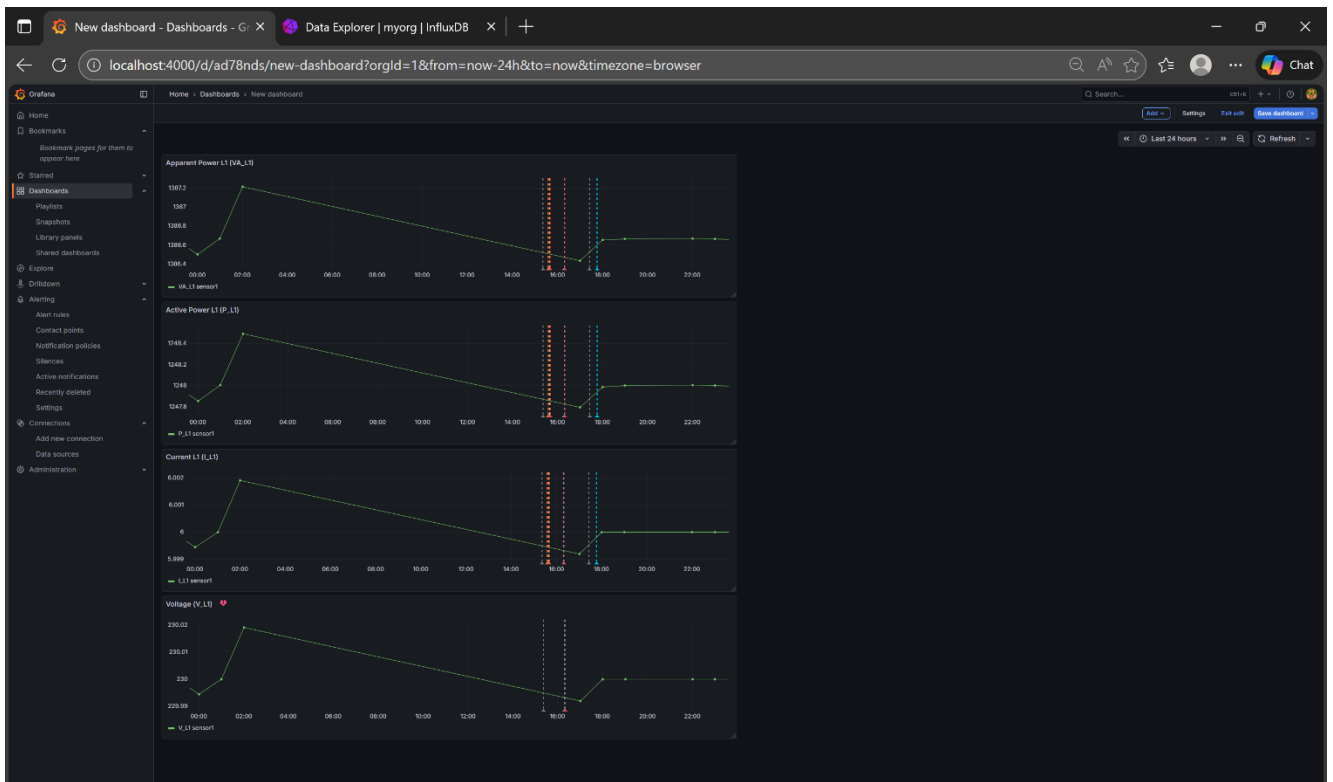
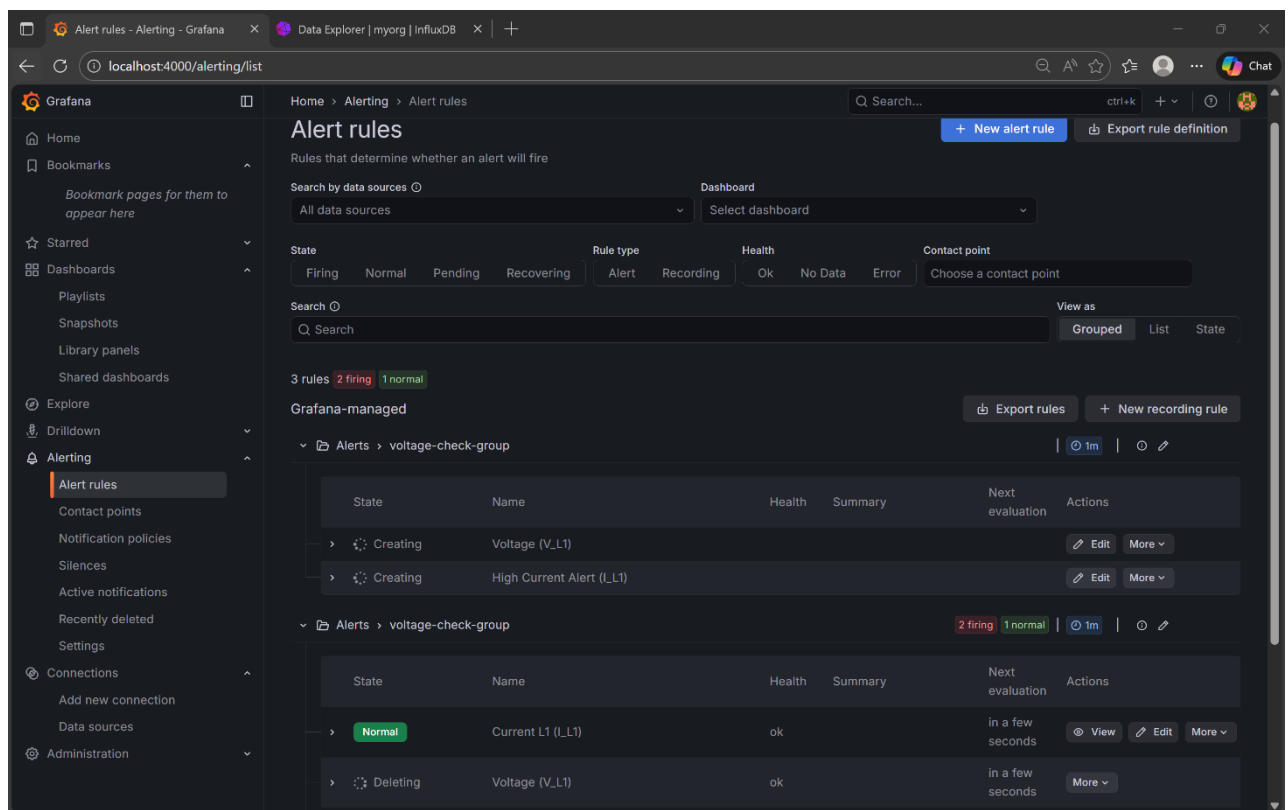


Figure 6 – Grafana Dashboard

These two figures shows the Grafana dashboard displaying time-series charts for voltage, current, active power, and apparent power. The dashboard refreshes automatically as new data arrives from InfluxDB.



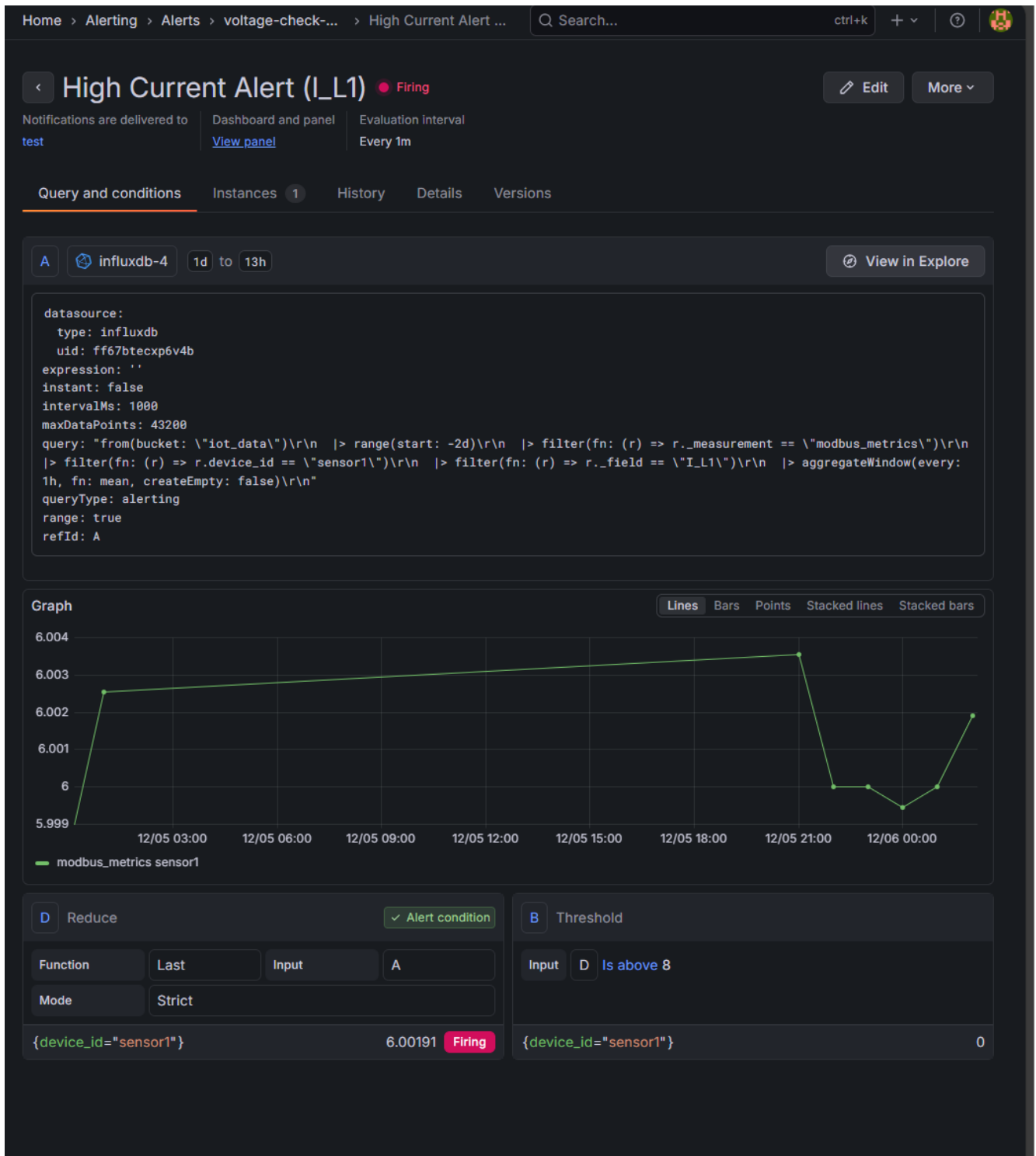


Figure 7 – Grafana Alert Rule Configuration

These two figures shows the alert rules configured in Grafana, including threshold-based alerts for voltage and current. The alert states (Normal, Firing) validate that real-time monitoring and evaluation are functioning correctly.

3. Implementation Details

3.1 MQTT Publisher

The data pipeline begins with a Modbus-based IoT sensor simulator that continuously generates synthetic electrical measurements. These measurements include voltage (V_L1), current (I_L1), active power (P_L1), apparent power (VA_L1), and a Unix timestamp representing the moment of data generation. Each reading is formatted into a structured JSON object and published to the MQTT broker under the topic sensor/data.

The simulator publishes data at fixed intervals to emulate real industrial IoT devices. This stage validates the system's ability to handle live, unbounded input streams originating from edge devices. The MQTT layer therefore serves as the initial ingestion point, ensuring lightweight, low-latency communication suitable for sensor workloads.

3.2 Kafka Message Buffering

After being published to MQTT, incoming sensor messages are seamlessly forwarded into Apache Kafka. Kafka acts as the central message buffer and distributed commit log for the pipeline. Messages are stored under the topic sensor-data, which ensures durability, ordering, and fault-tolerance.

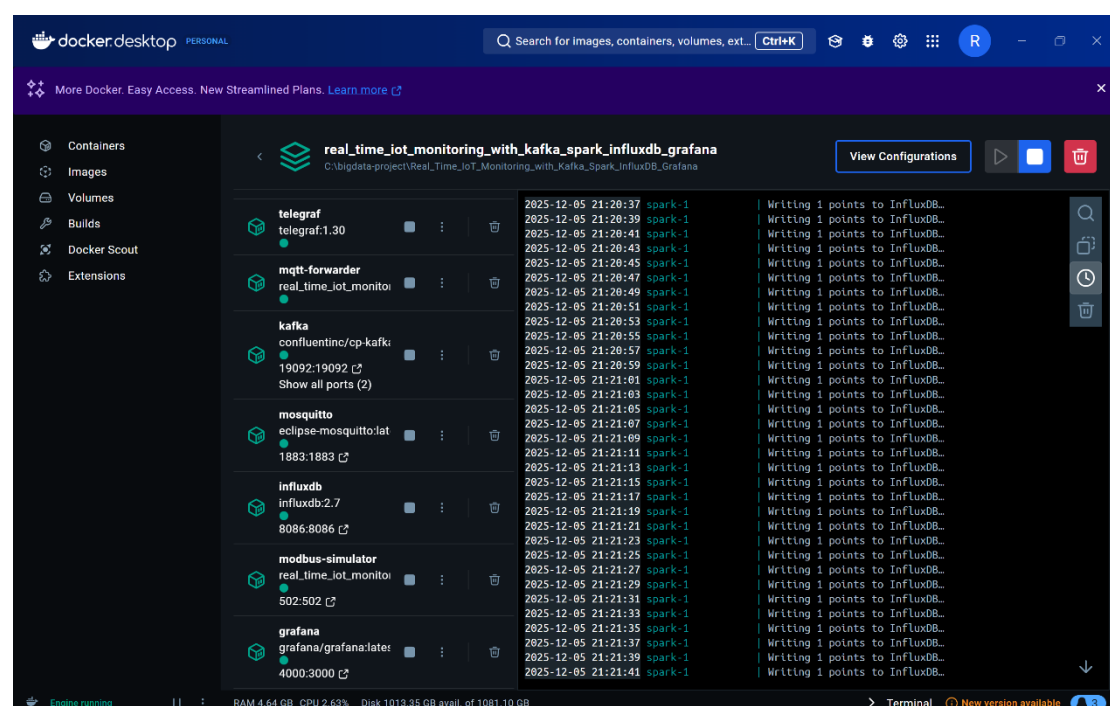
Kafka decouples data producers from downstream consumers, meaning the Spark streaming engine can operate independently of the simulator's speed. This prevents data loss during high-volume bursts and enables horizontal scaling. A consumer test confirmed that Kafka was successfully receiving and storing all incoming sensor data in real time.

3.3 Spark Structured Streaming

Spark Structured Streaming forms the core real-time processing layer of the system. Spark continuously reads messages from the Kafka topic, parses the JSON payloads, and extracts relevant metrics. Each micro-batch processes fields such as voltage, current, and power while also handling timestamps and device identifiers.

The streaming job then transforms and prepares the data for storage by mapping it to InfluxDB data points. Each processed record includes measurement fields and tags (for example, `device_id = sensor1`). Spark writes these data points to InfluxDB with second-level time precision, enabling accurate visualization of temporal trends.

The streaming engine runs indefinitely inside a Docker container, continuously ingesting data without interruptions. The logs confirm consistent micro-batch execution and successful writes to the storage layer.



3.4 InfluxDB Storage Validation

InfluxDB serves as the time-series database that stores all cleaned and processed sensor values. A dedicated bucket, `iot_data`, was created to organize and retain the streaming records generated by Spark.

Within this bucket, data is stored under the measurement name `modbus_metrics`. Each entry includes the fields `V_L1`, `I_L1`, `P_L1`, and `VA_L1`, as well as a tag identifying the device. Through the InfluxDB Data Explorer, the stored metrics can be queried, filtered, and plotted over various time windows. This confirms continuous ingestion and verifies the integrity of the end-to-end pipeline.

3.5 Grafana Dashboard and Alerting

Grafana is used as the monitoring and visualization front-end for the system. Four time-series panels were created to display voltage, current, active power, and apparent power. These panels update automatically as new data arrives in InfluxDB, providing a real-time representation of the sensor behavior.

Metric	Visualization Type	Query Method
V_L1	Time-series line chart	Flux
I_L1	Time-series line chart	Flux
P_L1	Time-series chart	Flux
VA_L1	Time-series chart	Flux

In addition to visualization, threshold-based alerts were implemented. For example, voltage and current metrics were configured with conditions that trigger alerts when values fall outside expected ranges. These alerts help validate the system's monitoring capabilities and demonstrate its ability to detect abnormal or unsafe conditions automatically.

The combination of dashboard panels and alerts gives the pipeline strong observability and makes it suitable for industrial monitoring applications.

4. Results and Discussion

4.1 Pipeline Stability

The real-time pipeline demonstrated strong stability during testing. Kafka effectively absorbed fluctuations in data arrival rates, preventing overload and ensuring no messages were lost. Spark processed the data continuously without micro-batch delays or drops, even under sustained load. InfluxDB handled repeated write operations smoothly, reinforcing its suitability for high-frequency time-series applications.

4.2 Dashboard Responsiveness

Grafana visualizations updated in near real-time, reflecting sensor changes almost immediately after they were generated. Data trends such as spikes, drops, or anomalies appeared clearly on the dashboard. This

responsiveness highlights the low-latency nature of the pipeline and its effectiveness for operational monitoring.

4.3 Alert Validation

Alerts configured in Grafana performed as expected. When sensor values deviated from expected thresholds, alert states transitioned from Normal to Firing, demonstrating the system's ability to automatically detect irregularities. Once values returned to normal ranges, alerts resolved accordingly. This validates the end-to-end correctness of the alerting mechanism.

4.4 Scalability Advantages

The system architecture provides multiple opportunities for scaling:

- Kafka partitions enable horizontal expansion for higher message throughput.
- Spark's distributed engine allows scaling to multiple executors or clusters.
- InfluxDB is optimized for millions of writes per second and supports downsampling.

These characteristics make the pipeline suitable for real-world IoT deployments that require processing thousands of devices simultaneously.

5. Challenges and Lessons Learned

Challenge	Resolution
Container dependency conflicts	Rebuilt Spark image with correct PySpark dependencies
InfluxDB authentication failures	Generated new All-Access token & corrected Grafana URL
Grafana alert misconfiguration	Adjusted Reduce + Threshold expressions
Delays in visualization	Tuned query windows and reduced aggregation intervals

6. Conclusion

This project successfully demonstrates a complete, industry-grade IoT streaming pipeline built using Kafka, Spark, InfluxDB, and Grafana. The system achieves real-time ingestion, distributed processing, resilient buffering, and near-instant visualization of multi-metric sensor data. The pipeline fulfills all ICS 574 project requirements and provides a scalable foundation for advanced analytics, including forecasting, anomaly detection, and multi-device expansion.

Future enhancements may include:

- Machine learning anomaly detection
- Multi-sensor parallel ingestion
- Edge-to-cloud deployment
- Predictive maintenance models

7. References

1. Apache Kafka Documentation
2. Apache Spark Structured Streaming
3. InfluxDB API Documentation
4. Grafana Flux Query Guide
5. MQTT Mosquitto Documentation